

# Making RestroOS work offline

What has to be built, what has to change in the existing system, and what can never work without a network — sequenced so each phase ships something usable.

Against build 30964dc · Target: a till that keeps billing with the internet unplugged

## START FROM THE GOOD NEWS

Your inventory design is **already offline-compatible**. The ledger records *deltas* (–4g Tea Leaves) in append-only rows rather than overwriting an absolute balance. Two terminals working offline and syncing later produce two rows that simply add up — no conflict, no lost update. Most systems have to rebuild their stock model for offline. You do not.

## Contents

|   |                                       |    |                                         |
|---|---------------------------------------|----|-----------------------------------------|
| 1 | What "offline-first" has to mean here | 7  | What cannot work offline                |
| 2 | Runtime: what the app runs inside     | 8  | Changes required in the current backend |
| 3 | The local database                    | 9  | Clock, auth and data at rest            |
| 4 | Identity and bill numbering           | 10 | How to test it                          |
| 5 | The sync engine                       | 11 | Phased delivery                         |
| 6 | Conflict resolution, per data type    | 12 | Effort and principal risks              |

## 1 What "offline-first" has to mean here

"Offline support" is often sold as a cache. That is not sufficient for a till. There are three distinct levels, and only the third is worth calling offline-first.

| LEVEL | CAPABILITY                                                                                                          | VERDICT FOR A RESTAURANT                                                       |
|-------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| L1    | Read-only cache — the menu loads offline, nothing can be written                                                    | Useless. Staff can look at prices but cannot take money.                       |
| L2    | Queue writes — orders captured locally and replayed on reconnect, but the UI still assumes a server                 | Fragile. Works until a second terminal, a restart, or a conflict.              |
| L3    | <b>Local-first</b> — the local database is the source of truth for the shift; the server is a replica it syncs with | <b>This is the target.</b> The app does not know or care whether it is online. |

## THE TEST THAT DEFINES DONE

Unplug the router mid-service. Take three orders on two terminals, print kitchen tickets, settle one in cash, void another. Restart one terminal. Plug the router back in. Everything appears on the server exactly once, stock is correct, and nobody on the floor noticed.

## 2 Runtime: what the app runs inside

A browser tab cannot reach a thermal printer or a cash drawer, and its storage is evictable by the OS. The runtime has to change.

| OPTION               | PRINTER / DRAWER | STORAGE              | ASSESSMENT                                                                                                                                                                                          |
|----------------------|------------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PWA + service worker | No               | IndexedDB, evictable | Cheapest, and reuses the current React code — but cannot drive hardware, and the browser may evict data. Not viable as the end state.                                                               |
| Electron             | Yes              | SQLite on disk       | <b>Recommended.</b> Keeps most of your React code, gives real filesystem access, node-level USB/LAN printing, and a mature update mechanism. Runs on the Windows terminals restaurants already own. |
| Tauri                | Yes              | SQLite on disk       | Much smaller binaries and lower memory than Electron, but the Rust side is a new skill on the team and the printing ecosystem is thinner.                                                           |
| React Native         | Partly           | SQLite               | Right answer for Android tablet POS specifically, which is common in India. But it is a full front-end rewrite — no React DOM reuse.                                                                |

### RECOMMENDATION

**Electron on Windows first.** It preserves the largest amount of what you have already built, and the reference site already runs a Windows terminal. Treat an Android build as a later, separate decision — not a simultaneous one.

## 3 The local database

SQLite via `better-sqlite3`, running in the Electron main process. Synchronous, transactional, and fast enough that reads feel instant — which is the entire point.

### Tables to hold locally

| GROUP                                | DIRECTION  | NOTES                                                               |
|--------------------------------------|------------|---------------------------------------------------------------------|
| Menu, categories, options, recipes   | Pull only  | Reference data. Server is authoritative; the device never edits it. |
| Tables, branches, staff, permissions | Pull only  | Same.                                                               |
| Orders, order items, payments        | Push only  | Created on the device. The device owns them.                        |
| Inventory transactions               | Push only  | Append-only deltas. Merges without conflict.                        |
| Shifts, cash counts                  | Push only  | Device-owned for the duration of the shift.                         |
| Outbox                               | Local only | The queue of unsynced operations. Never leaves the device.          |

The split matters: **anything the device only pulls can never conflict**, and anything it only pushes conflicts only with itself. Designing the boundary this way removes most of the hard cases before they exist.

## 4 Identity and bill numbering

### THE PROBLEM THAT BREAKS NAÏVE IMPLEMENTATIONS

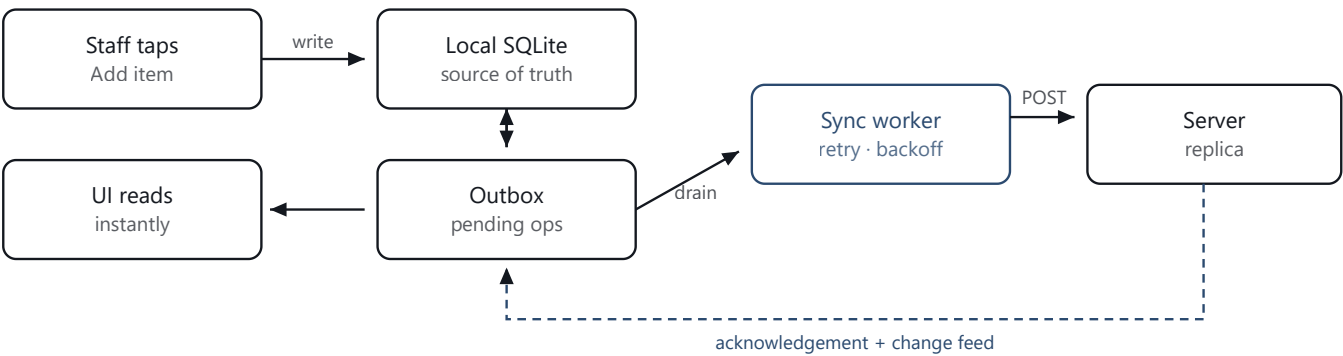
Your `orderId` is a database auto-increment. Offline, the device cannot ask the database for the next number. Two terminals both take an order, both guess `#301`, and on reconnect one silently overwrites the other or the insert fails. This must be solved before anything else is built.

### Solution — two separate identifiers

| IDENTIFIER                                      | RULE                                                                                                                                                                                                                                                   |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Primary key</b><br><code>OrderUuid</code>    | A UUID generated on the device. Globally unique with no coordination. This is what sync, references and idempotency use.                                                                                                                               |
| <b>Human bill number</b><br><code>BillNo</code> | Allocated from a <i>block</i> reserved per device while online — terminal A holds 3000–3999, terminal B 4000–4999. Sequential and readable for customers and GST, with no risk of collision. The device requests a fresh block when it is running low. |

This is also a prerequisite for per-tenant GST-compliant invoice numbering, which the launch-readiness register already lists separately — the two should be built together, once.

## 5 The sync engine



*The write path. The UI never waits on the network: a tap commits to local SQLite and an outbox row in the same transaction, and the sync worker drains that queue independently.*

### The outbox pattern

Every mutation writes *two* things inside one local transaction: the change itself, and a row in `outbox` describing it. If the app crashes between them, neither happened. The sync worker then drains the outbox in order, with exponential backoff.

```
outbox
  id          local autoincrement, preserves ordering
  op_uuid     idempotency key sent to the server
  entity      'order' | 'payment' | 'inventory_txn' | 'shift'
  entity_uuid the record it refers to
  payload     JSON
  attempts    retry counter
  last_error  for support and diagnostics
```

### Idempotency is mandatory

The device will retry after a timeout it cannot distinguish from a failure. The server must accept `op_uuid` and treat a repeat as a no-op returning the original result. Without this, a flaky connection produces duplicate orders and double stock deduction.

#### YOU HAVE ALREADY DONE THIS ONCE

`InventoryService.consumeForOrder` guards itself with a row-locked check for existing rows against the order, so calling it twice cannot double-deduct. That is exactly the property every sync endpoint now needs. The pattern is proven in your codebase — it needs generalising, not inventing.

### Pulling changes down

The device asks `GET /sync/changes?since=<cursor>` and receives menu, price and staff updates. Reference data is versioned so a stale device can detect it has fallen behind. Prices must be **stamped onto the order line at capture time**, never looked up later, so a bill reprinted next week still shows what the customer actually paid.

## 6 Conflict resolution, per data type

Conflicts are only hard where two writers can touch the same thing. Most of your data has exactly one writer.

| DATA                   | CONFLICT RISK      | RESOLUTION                                                                                                                                                                        |
|------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Menu, prices, recipes  | None               | Pull only. Server always wins.                                                                                                                                                    |
| Orders and items       | None               | Device-owned, UUID keyed, append only. Two devices cannot create the same order.                                                                                                  |
| Stock levels           | None               | <b>Already solved by your ledger design.</b> Deltas commute — two offline terminals each writing −4g produce −8g when merged, which is correct.                                   |
| Table occupancy        | Real               | Two terminals can both open table A2 while offline. Resolve by merging both orders onto the table and flagging for staff, rather than discarding one. Never silently drop a bill. |
| Coupon usage limits    | Unsolvable offline | "Once per customer" cannot be enforced without a shared counter. See §7.                                                                                                          |
| Item 86 / availability | Real               | Last-write-wins with a timestamp is acceptable; the cost of a brief disagreement is low.                                                                                          |

## 7 What cannot work offline

Being explicit about this is what separates a workable design from one that fails in front of a customer.

| CAPABILITY                    | OFFLINE       | HANDLING                                                                                                                                                                                            |
|-------------------------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cash payment                  | Works         | Fully local. This is the majority of a chai counter's volume.                                                                                                                                       |
| Card / UPI via Razorpay       | Cannot        | Requires the gateway. Offline, the terminal must record the payment as taken on an external machine and reconcile later — the UI has to make that state explicit, never pretend it captured online. |
| Coupon with usage limits      | Cannot verify | Either block limited coupons offline, or accept the over-redemption risk deliberately and flag it on sync. This is a business decision, not a technical one.                                        |
| New customer lookup           | Degraded      | Search a locally cached subset; unknown numbers become a new local customer, deduplicated server-side on sync.                                                                                      |
| Online orders from storefront | Cannot        | They arrive over the network by definition. They queue server-side and appear when the terminal reconnects.                                                                                         |
| Live cross-branch reporting   | Cannot        | Reports show data as of last sync, and must say so on screen.                                                                                                                                       |

## 8 Changes required in the current backend

The server survives largely intact. These are the specific changes it needs.

| CHANGE | EFFORT | DETAIL |
|--------|--------|--------|
|--------|--------|--------|

|                                                                             |          |                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>UUID primary keys</b> on orders, items, payments, inventory transactions | 1–2 wks  | Keep existing integer IDs as secondary for now; add a UUID column with a unique index and migrate references. The hardest single change on this list.                                                                                                                                                                                            |
| <b>Idempotent write endpoints</b>                                           | 1–2 wks  | Accept <code>op_uuid</code> , store it, return the original result on repeat. Generalise the pattern already used in <code>consumeForOrder</code> .                                                                                                                                                                                              |
| <b>Bill-number block allocation</b>                                         | 3–4 days | <code>POST /sync/allocate-bill-block</code> handing a device a reserved range.                                                                                                                                                                                                                                                                   |
| <b>Change feed</b>                                                          | 1 wk     | <code>GET /sync/changes?since=</code> with a monotonic cursor. Requires <code>UpdatedAt</code> maintained reliably on reference tables.                                                                                                                                                                                                          |
| <b>Move pricing and tax into shared code</b>                                | 1–2 wks  | Today <code>resolveMenuItemOptions</code> and <code>roundGst</code> run only on the server, deliberately, so the client is never trusted. Offline, the device <i>must</i> compute the total. Extract these into a package both sides import, and have the server <b>re-verify on sync and flag any mismatch</b> rather than trusting the device. |
| <b>Accept backdated orders</b>                                              | 3–4 days | An order synced at 21:40 may have occurred at 19:15. Timestamps must come from the payload, with the server recording both event time and receipt time.                                                                                                                                                                                          |
| <b>Long-lived device sessions</b>                                           | 3–4 days | A seven-day JWT expiring mid-shift with no connectivity locks staff out. Devices need a separate credential that renews on sync.                                                                                                                                                                                                                 |

### THE SECURITY TRADE-OFF YOU ARE ACCEPTING

Your current design re-prices every order server-side and explicitly does not trust the client. Offline capability breaks that guarantee — the device must be trusted to compute a total. The mitigation is not to abandon verification but to move it: the server recomputes on sync and raises a discrepancy for a human, rather than silently accepting or silently rejecting.

## 9 Clock, auth and data at rest

---

### Clock skew

A terminal with a wrong clock produces orders timestamped in the wrong hour, which corrupts sales-by-hour reporting and GST periods. On every sync, record the offset between device and server time and store both the device time and the corrected time on each record.

### Authentication during an outage

Staff must be able to sign in with no network. Cache a per-staff PIN hash locally, refreshed on sync. This dovetails with the PIN model a POS needs anyway, and with per-staff order attribution already on the launch-readiness list.

### Data at rest

The device will hold customer names, phone numbers and order history in a local file. A stolen terminal is a data breach. Encrypt the database (SQLCipher or OS-level), and hold only a recent working window locally rather than full history.

## 10 How to test it

---

Offline bugs do not appear in normal development, because development always has a network. Testing has to be deliberate.

1. **Airplane-mode suite.** Automated runs with the network forced down: take orders, settle, restart the app, restore the network, assert the server state matches exactly and nothing duplicated.
2. **Two-terminal conflict tests.** Both devices offline, both touching the same table, both consuming the same ingredient. Assert the documented resolution actually happens.
3. **Chaos on the sync path.** Kill the process mid-drain, respond with 500s, respond with a timeout after the server has already committed. The last one is the case idempotency exists for and the one most often missed.
4. **Clock-skew tests.** Set the device clock hours out and confirm reports still bucket correctly.
5. **Long-outage soak.** Eight hours offline with a full day's order volume, then sync. Watch memory, outbox depth and sync duration.

## 11 Phased delivery

---

Each phase ships something usable. Do not attempt this as one release.

### Phase A — Foundations (server-side, 3–4 weeks)

UUID keys · idempotent endpoints · shared pricing/tax package · change feed · bill-block allocation. *Nothing visible ships, but every later phase depends on it.* Existing web app continues working unchanged throughout.

### Phase B — Electron shell with local cache (3–4 weeks)

Package the existing React POS into Electron. SQLite holds pulled reference data, so **the menu works offline**. Writes still require a network. Add the local ESC/POS print agent here — printing is valuable on its own and independent of sync.

### Phase C — Offline order capture (4–6 weeks)

Outbox, sync worker, local order creation, offline cash settlement. This is the phase that delivers the actual promise. Ship to **one** friendly site and watch it closely.

### Phase D — Full local-first (4–6 weeks)

Shifts and cash reconciliation offline · KOT lifecycle local · table-conflict merging · encryption at rest · PIN auth.

### Phase E — Hardening (3–4 weeks)

The full test matrix in §10, an auto-update channel, and remote diagnostics for support — you will need to see a terminal's outbox depth without visiting the restaurant.

## 12 Effort and principal risks

| PHASE                | EFFORT    | PRINCIPAL RISK                                                                                                                  |
|----------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------|
| A — Foundations      | 3–4 weeks | The UUID migration touches every table with a foreign key. Do it on staging first, and only once staging exists.                |
| B — Shell + printing | 3–4 weeks | Printer driver variation across models is the classic time sink. Standardise on one or two printers you have physically tested. |
| C — Offline capture  | 4–6 weeks | Duplicate orders from retry storms. Idempotency must be correct before this ships, not after.                                   |
| D — Full local-first | 4–6 weeks | Table-conflict merging is where correctness is easiest to get subtly wrong. Never resolve by discarding a bill.                 |
| E — Hardening        | 3–4 weeks | Under-scoped in most plans, then discovered in production.                                                                      |

**Total: 17–24 weeks** for one developer, or roughly 10–14 with two working in parallel from Phase B onward.

### TWO PREREQUISITES BEFORE PHASE A STARTS

**A staging environment**, because the UUID migration cannot be validated on production. And **tests on the billing screen**, because you are about to change how every order is created, and today that screen has no automated coverage at all. Both are already blockers on the launch-readiness register; offline work should not begin until they exist.

Plan against build `30964dc`. Estimates assume developers familiar with the codebase and exclude hardware procurement and pilot-site time. The recommendation to run Electron on Windows first is specific to the reference site; an Android tablet build is a separate decision with different trade-offs.